

Multi-Client Chat Application using TCP/SSL

Introduction

This project aims to create and develop a client-server text-based chat system using the TCP protocol to ensure reliable communication between users. It also allows for the addition of SSL security to encrypt the connection and prevent unauthorized access. The system enables multiple users to communicate with each other via a single central server capable of receiving multiple simultaneous connections using multithreading technology.

Project Overview:

This project provides a chat environment between multiple users and a central server, allowing users to send public and private messages to specific users. The server receives these messages and, based on their type, manages and routes them. The system allows login and user display, and can also integrate SSL to secure data exchange and protect messages and passwords during transmission.

Project Goals:

- 1-Creating a server capable of collaborating with multiple users simultaneously
- 2-Using the TCP protocol to ensure reliable message transmission without data loss
- 3-Support for sending public messages to all users
- 4-Support for sending private messages to a specific user
- 5-Providing a simple login system that verifies usernames
- 6-User list management
- 7-Activate SSL for data encryption

Project Scope:

Project includes:

- 1-A server running using SSLServerSocket
- 2-A client that works using SSLsockets
- 3-Support for multiple connections using Multithreading.
- 4-Sending and receiving text messages
- 5-Public messages (Broadcast).
- 6-Private messages.
- 7-Log in and verify usernames
- 8-Handling user logout and connection interruption

The project does not include:

- 1-graphical user interface.
- 2-real database
- 3-File transfer
- 4-Support for non-text languages

System Functional Requirements (NFRs):

Server Functional Requirements:

- 1-The server must accept multiple clients.
- 2-The server must create a separate thread for each client.
- 3-The server must receive text messages and identify their type.
- 4-The server must direct general messages to all clients.
- 5-The server must forward private messages only to the specified recipient.

- 6-The server must maintain a list of connected users' names.
- 7-The server must remove the client upon exit or loss of connection.
- 8-SSL must be supported via SSLServerSocket.

Client Functional Requirements:

- 1-The client must connect to the server via TCP or SSL.
- 2-The customer must send their username upon login.
- 3-The customer must be able to send public messages.
- 4-The customer must be able to send messages in the specified format.
- 5-The client must receive messages from the server immediately.
- 6-The client must display server messages on the user screen.

Non-Functional Requirements (NFRs)

1. Performance

- The server must support **at least 5–20 concurrent clients** without noticeable delay.
- Message delivery (broadcast or private) should occur within **1 second** under normal network conditions.
- The server must handle multiple threads efficiently without causing CPU overload.

2. Reliability

- The system should continue operating even if one or more clients disconnect unexpectedly.
- The server should detect client termination and clean up resources immediately.
- Message processing should be consistent and follow the defined communication protocol.

3. Availability

- The server should remain active as long as the application is running.
- A failure in one client must not affect the availability of the server or other clients.

4. Security

- The system should prevent unauthorized users from joining by requiring a username on connection.
- All message data should be kept private between clients (optional SSL/TLS can be added for secure transmission).
- The server should validate all incoming messages to avoid malformed commands.

5. Usability

- The client interface should be simple, text-based, and easy to understand.
- Commands for sending broadcast and private messages should be intuitive (e.g., /pm username message).
- System responses should be clear and human-readable.

System Features:

Authentication:

This feature allows each client to authenticate before joining the chat system. The client must provide a unique username. The server checks whether the username is already used; if it is, the server rejects the login request. Once authenticated, the user is added to the list of active users.

What it includes:

Unique username check

Accept/Reject login

Adding user to active users list

Denying duplicate usernames

Messaging:

This feature enables communication between clients through the server. All messages are sent from clients to the server, and the server determines how to distribute them.

Types of messaging supported:

Broadcast messaging: Send a message to all connected clients.

Private messaging: Send a message to a specific user.

Server system messages: Notifications when a user joins or leaves.

What it includes:

Sending messages to the server

Server routing messages to target client(s)

Handling message formatting and protocol

File Transfer:

Not supported

Communication Methods:

✓ TCP Communication:

The system uses the TCP protocol as the main communication method between the server and the clients. TCP provides a reliable, connection-oriented communication channel, ensuring that all chat messages and file data are delivered correctly and in order.

How TCP is used in the system:

1-The server opens a ServerSocket on a specific port

2-Each client connects using a Socket

3-Communication happens through Input/Output Streams

4-The connection remains active throughout the chat session

5-All messaging and file transfer operations are sent through TCP streams

2-SSL Communication (Secure Communication over TCP):

To enhance security, the system optionally supports SSL/TLS, which provides encrypted communication over the existing TCP connection.

SSL adds:

Confidentiality : prevents eavesdropping

Integrity : prevents message tampering

Authentication :server identity verification

How SSL is used:

1-The server uses SSLServerSocket instead of ServerSocket

2-Clients connect using SSLSocket

3-All messages and files are encrypted automatically

4-SSL certificates (keystore) are used for secure handshake

Risks

1. Thread mismanagement causing blocking or performance issues.
2. Unexpected client disconnections leading to exceptions or hanging threads.
3. Malformed or malicious client messages breaking the communication protocol.
4. Performance degradation with many simultaneous clients.
5. Network latency affecting message delivery.
6. Challenges integrating SSL/TLS security.

7. Lack of encryption (if SSL not used) leading to security vulnerabilities.

Constraints

1. TCP protocol limitations (slower than UDP, server-based broadcast only).
2. Text-based console interface restricts user interaction.
3. Single server dependency — system stops if the server stops.
4. No database or persistent storage.
5. Limited system resources limit number of clients.
6. Restricted to Java Sockets without advanced libraries or frameworks.

Teamwork:

Noor Aldaaja

Mira Almuhsen

Layan Amjad

